CSED233 · 데이터구조 · 포스텍

Ch8. Sorting

1. Sorting 기본 정의

핵심 개념

- **정렬**: 원소 (key, info) 쌍들을 특정 순서로 재배치
- **Linear ordering** 조건:
  - **Trichotomy**: 임의 a, b에 대해 $a < b, a = b, a > b$ 중 **정확히 하나** 성립
  - **Transitivity**: $a < b$ & $b < c \rightarrow a < c$

분류 축

축	종류
메모리	Internal (in-memory) / External (disk)
방법	Comparison / Non-comparison
안정성	Stable / Unstable

Stability(안정성) 정의

- **Stable**: 동일 key 원소들의 **원래 상대 순서 보존**
- 예: $(3, y)(3, x)(4, a)(5, b)$ 정렬
  - Stable → $(3, y)(3, x)(4, a)(5, b)$ (y가 x 앞, 원래 순서)
  - Unstable → $(3, x)(3, y)(4, a)(5, b)$ (바뀜)
- **왜 중요한가**: 다중 키 정렬 시 (예: 학번 정렬 후 점수 정렬) 이전 정렬 결과 유지

2. Bubble Sort(버블 정렬)

아이디어

- **k-th pass에서 k-th 최솟값이 k-th 위치로 '떠오름'**
- 인접 원소 비교 → 순서 뒤바뀐 쌍 swap
- **Early termination**: 한 pass에서 swap 0번 → 이미 정렬됨 → 종료

복잡도

Case	시간
Best (이미 정렬)	$O(n)$ — early termination
Average / Worst	$O(n^2)$ — $n(n-1)/2$ 비교, $O(n^2)$ swap

- **용도**: $n < 100$ 정도 소규모만. 실무 거의 안 씀.

3. Selection Sort(선택 정렬)

아이디어

- **k-th pass에서 unsorted 부분의 최솟값 찾아 $A[k]$와 swap**
- **pass당 swap 최대 1번** → swap 비용 중요한 상황에 유리

복잡도

- **비교 횟수는 항상 $O(n^2)$**: best = average = worst
  - 이유: 매 pass마다 나머지 전체를 스캔해 최솟값 탐색
- **Swap 횟수는 $n-1$번** (bubble보다 훨씬 적음)

Bubble vs Selection

	Bubble	Selection

Best case	$O(n)$	$O(n^2)$
Swap count	$O(n^2)$	$O(n)$
Stable?	Yes	No (일반적)

4. Insertion Sort(삽입 정렬)

아이디어

- **k-th pass에서 $A[k]$를 이미 정렬된 $A[1..k-1]$에 올바른 위치에 삽입**
- 카드 정렬 직관: 한 장씩 뽑아 손에 있는 정렬된 카드에 꽂음

복잡도

- **Best** $O(n)$: 이미 정렬된 경우 → 각 원소 비교 1번
- **Average / Worst** $O(n^2)$: $\Theta(n^2)$ swap

실전 가치

- **작은 n (< 15)에선 Merge/Quick보다 빠름** (상수항 작음)
- → **Merge/Quick sort에서 recursion base로 사용** (hybrid 기법)

5. Bucket Sort (Non-Comparison)

아이디어

- 키 값으로 **직접 index** 접근 → 비교 없이 분류
- **Phase 1 (Scatter)**: 각 원소를 key에 해당하는 bucket에 배치
- **Phase 2 (Gather)**: bucket 순서대로 꺼내 연결

복잡도

- Scatter: $O(n)$
- Gather: $O(n + N)$ (bucket 수 N 고려)

- **Total**: $O(n + N)$

적용 조건

- **Key 범위** $[0, N-1]$**이 작을 때** 효율적
- $N \gg n$이면 **오히려 느림** (빈 bucket 스캔)

Q1. Float key(예: 0.78, 0.12, …)도 bucket sort가 가능한가?

> **힌트**

bucket sort는 key의 값 범위를 구간으로 나누는 방식이다.

> **정답**

개념은 힌트 참고.

- **가능**. 구간 $[0, 1)$을 k개 bucket으로 분할 (예: $[0, 0.1), [0.1, 0.2), \ldots$) → bucket 내부는 insertion sort 등으로 재정렬. **uniform 분포 가정 시** 평균 $O(n)$.

Q2. Bucket sort가 느려지는 경우는?

> **힌트**

bucket의 개수와 분포 균일성이 성능을 좌우한다.

> **정답**

개념은 힌트 참고.

- (1) Key 범위 N이 n에 비해 너무 클 때, (2) 한 bucket에 몰릴 때 (skewed distribution)

6. Merge Sort(병합 정렬)

Divide & Conquer

1. **Divide**: 배열을 반으로 분할
2. **Conquer**: 각 반을 재귀적으로 정렬
3. **Combine**: 두 정렬된 리스트를 **merge**

Merge 연산 자체

- 두 정렬 리스트 A, B를 merge
- **두 포인터**로 맨 앞 원소 비교 → 작은 쪽을 output에 추가
- 한쪽 소진 시 나머지 그대로 append
- **시간:** $O(n+m)$

재귀 트리 & 복잡도 유도

- Level 수: $O(\log n)$
- 각 level에서 **merge 총 작업량 =** $O(n)$ (모든 원소 한 번씩)
- **Total =** $O(n \log n)$

단점 & 최적화

- **추가 메모리** $O(n)$ **필요** (in-place 어려움)
- **Stable** ✓
- **Worst case도** $O(n \log n)$ ✓ → 보장 필요 시 유리
- 최적화: $n \leq 15$일 때는 insertion sort 사용 (상수항 작음)

Non-recursive (Bottom-up)

- Size 1 segment에서 시작 → pairwise merge → size 2 → size 4 → ...
- Downward pass 제거, 직접 위로 병합

7. Quick Sort(퀵 정렬)

Divide & Conquer (Partition 기반)

1. **Pivot(피벗) 선택**
2. **Partition**: pivot 기준 3 그룹
    - Left: key < pivot
    - Pivot
    - Right: key ≥ pivot

3. **재귀**: left, right 각각 정렬
4. **Combine**: `sorted(L) + pivot + sorted(R)` (concat 자체는 거의 free)

Partition 방법

Method 2 (In-place)

- 두 포인터 L, R (양 끝에서 시작)
- L: pivot 이상인 값 탐색 (왼→오)
- R: pivot 이하인 값 탐색 (오→왼)
- L, R 서로 다른 쪽이면 swap, 마주치면 종료 → pivot(첫 원소)과 R 위치 swap
- **추가 메모리 거의 없음** → cache friendly

Pivot 선택 전략

전략	특징
First/Last	간단. **정렬된 입력에** $O(n^2)$
Middle	일부 경우에 잘 동작
Random	평균 좋으나 난수 생성 비용
Median-of-three	first, middle, last 중 중간값 → 안정적

복잡도 유도

점화식

$$t(n) = t(|L|) + t(|R|) + d \cdot n$$

Best / Average ($|L| \approx |R| \approx n/2$):

$$t(n) = 2t(n/2) + dn = O(n \log n)$$

Worst ($|L| = 0$ 또는 $|R| = 0$, 예: 이미 정렬된 입력 + leftmost pivot):

$$t(n) = t(n-1) + dn = O(n^2)$$

8. Quick vs Merge vs Heap(힙)

비교

	Quick	Merge	Heap
Best	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Average	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Worst	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Extra space	$O(\log n)$ (stack)	$O(n)$	$O(1)$
In-place	✓	✗	✓
Stable	✗	✓	✗

실전 선택 기준

- **Quick sort**: 평균 가장 빠름 (상수항 작음, cache-friendly) → **일반적인 경우 1순위**
- **Merge sort**: **worst-case 보장** + stable 필요 시 (예: 외부 정렬, linked list)
- **Heap sort**: **실시간 시스템** (worst $O(n \log n)$ 보장 + $O(1)$ 공간)
  - quick sort는 worst $O(n^2)$이라 실시간엔 위험

Quick space 주의: O(log n)은 **평균** 재귀 stack 깊이. skewed 분할이 계속되면 깊이 O(n) → 최악 공간도 O(n) (작은 쪽 먼저 재귀하면 최악도 O(log n) 유지 가능).

Quick의 cache 이점

- In-place → 같은 배열 접근 → **cache coherence**
- Merge sort는 별도 buffer 왕복 → cache miss 발생

Summary

Algorithm	Best	Avg	Worst	Space	Stable	Method
Bubble	n	n^2	n^2	1	✓	Adjacent swap
Selection	n^2	n^2	n^2	1	✗	Min-search
Insertion	n	n^2	n^2	1	✓	Insert into sorted
Merge	$n \log n$	$n \log n$	$n \log n$	n	✓	D&C (merge)
Quick	$n \log n$	$n \log n$	n^2	$\log n$	✗	D&C (partition)
Heap	$n \log n$	$n \log n$	$n \log n$	1	✗	Heap
Bucket	$n + N$	$n + N$	n^{2*}	$n + N$	✓	Non-comparison

* float key 변형처럼 bucket 내부 정렬(insertion sort)이 필요한 경우, 한 bucket에 몰리면 내부 정렬 비용 **Θ(n²)**이 지배. 정수 key 직접 index 버전은 worst에도 O(n+N).

- **Comparison sort의 하한**: $\Omega(n \log n)$ (결정 트리 이론)
- Bucket/radix는 비교 없이 $O(n)$ 가능 → 단, key 분포 조건 필요